Digital Image Processing
Spring 2025
Lab:6 Image smoothing and sharpening

Name	
Student ID	
Date	

Objectives

Objectives

- Understand and implement image filtering techniques: convolution, box filter, Gaussian filter, and unsharp masking.
- Explore how these filters affect image sharpness, blurring, and enhancement.

Prerequisites

Make sure the following Python libraries are installed:

- `opencv-python` (OpenCV)
- `matplotlib`
- `numpy`
- `scikit-image`

You can install them using:

You can install them using following command in command prompt or bash:

pip install opencv-python matplotlib numpy scikit-image

Digital Image Processing
Spring 2025
Lab:6 Image smoothing and sharpening

Part 1: Convolution

Convolution is a fundamental operation in image processing that applies a filter or kernel to an image. This operation modifies the image based on neighboring pixel values.

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

# Sample code for applying a convolution kernel
image = cv2.imread('image.png',
cv2.IMREAD_GRAYSCALE)
kernel = np.array([[1, 2, 1], [0, 0, 0], [-1, -2, -
1]])  # Example kernel (Sobel edge detection)
convolved_image = cv2.filter2D(image, -1, kernel)

plt.imshow(convolved_image, cmap='gray')
plt.show()
```

or alternatively, you can use the following function to create the box kernel of any size

```
def box_filt(n):
    """
    Creates a box filter kernel of size n x n.
    """
    if n <= 0 or n % 2 == 0:
        raise ValueError("Window size must be a
positive odd integer.")

    # Create a kernel with all elements equal to
1/(n*n)
    kernel = np.ones((n, n), np.float32) / (n * n)
    return kernel
```

instead of using the function cv2.filter2D, you can use user defined function for the convolution

Digital Image Processing
Spring 2025
Lab:6 Image smoothing and sharpening

```
def apply_filters(image_input, box, filt_size):
    pad_size = int(np.ceil(filt_size/2))

    image_padded = np.pad(image_input,
pad_width=((pad_size,pad_size),(pad_size,
pad_size)), mode='symmetric')
    image_box = copy.deepcopy(image_input)

    for i in range(row):
        for j in range(column):
            # image_input[i,j] = 0
            patch_curr = image_padded[i:i +
filt_size, j:j + filt_size]
            results_box = box * patch_curr
            image_box[i,j] = np.sum(results_box)


return image_box
```

Task 1:

1. Apply histogram equalization to an image and observe the effect on contrast.
2. Compare the histograms before and after equalization.
3. Discuss why histogram equalization improves contrast in images with poor lighting.

Digital Image Processing
Spring 2025
Lab:6 Image smoothing and sharpening

Part 2: Gaussian Filter

Theory:
A Gaussian filter smooths an image by averaging pixel values, with more weight given to pixels closer to the center of the kernel. It's used to reduce noise and blur.

Task 2:

1. Apply a Gaussian filter to the image using `cv2.GaussianBlur()`.
2. Experiment with different kernel sizes (e.g., 3x3, 5x5) and sigma values.
3. Compare the results to those from the box filter and describe the differences.
4. Display all results side by side.

```
# Applying a Gaussian filter
gaussian_filtered_image = cv2.GaussianBlur(image,
(5, 5), 0)

plt.imshow(gaussian_filtered_image, cmap='gray')
plt.title('Gaussian Filtered')
plt.show()
```

Alternatively, instead of using GaussianBlur to create Gaussian kernel, you can use the following user defined function as well

```
def gauss_filt(n, sigma=1.0):
    """
    Creates a Gaussian filter kernel of size n x n
with standard deviation sigma.
    """
    if n <= 0 or n % 2 == 0:
        raise ValueError("Window size must be a
positive odd integer.")

    # Create a Gaussian kernel
    ax = np.linspace(-(n - 1) / 2., (n - 1) / 2.,
n)
    xx, yy = np.meshgrid(ax, ax)
```

```
    kernel = np.exp(-0.5 * (np.square(xx) +
np.square(yy)) / np.square(sigma))

    # Normalize the kernel
    kernel = kernel / np.sum(kernel)
    return kernel
```

Part 3: Unsharp Masking

Theory:
Unsharp masking enhances edges by subtracting a blurred version of the image from the original, making it appear sharper.

Task 3:

1. Apply Gaussian blur to the image as part of unsharp masking.
2. Subtract the blurred image from the original to highlight edges.
3. Combine the original and edge-highlighted image using `addWeighted()`.
4. Adjust the weighting to see different sharpness levels.

```
blurred = cv2.GaussianBlur(image, (9, 9), 10)
unsharp_image = cv2.addWeighted(image, 1.5,
blurred, -0.5, 0)

plt.imshow(unsharp_image, cmap='gray')
plt.title('Unsharp Masking')
plt.show()
```

Alternatively, you can use the following source code for the unsharp masking as well

```
import copy

import cv2
import numpy as np
from matplotlib import pyplot as plt
```

```
def box_filt(n):
    """
    Creates a box filter kernel of size n x n.
    """
    if n <= 0 or n % 2 == 0:
        raise ValueError("Window size must be a
positive odd integer.")

    # Create a kernel with all elements equal to
1/(n*n)
    kernel = np.ones((n, n), np.float32) / (n * n)
    return kernel


def apply_filters(image_input, box, filt_size):
    pad_size = int(np.ceil(filt_size/2))
    # pad_size = pad_size.astype('uint8')
    # print(pad_size.dtype)
    image_padded = np.pad(image_input,
pad_width=((pad_size,pad_size),(pad_size,
pad_size)), mode='symmetric')
    image_box = copy.deepcopy(image_input)
    row, column = image_input.shape
    for i in range(row):
        for j in range(column):
            # image_input[i,j] = 0
            patch_curr = image_padded[i:i +
filt_size, j:j + filt_size]
            results_box = box * patch_curr
            image_box[i,j] = np.sum(results_box)
    return image_box

filt_size = 5
k = 2
```

```
image_input = cv2.imread('fruit basket.jpg',
0).astype('float32')/255
box = box_filt(filt_size)
image_blur = apply_filters(image_input, box,
filt_size)
image_diff = image_input - image_blur
image_sharp = image_input+k*image_diff
cv2.imshow('sharpen image', image_sharp)
cv2.waitKey(-1)
```

Task 4:

1. Using the source code provided above, use various combinations of parameters filt_size and k and write down the impact of their variations. Use various filt_size such as 3, 5, 7, 9, 11, 15 as well as various values of k such as -2, -1, -0.5, 0, 0.5, 1, 2, 3, 5, 9

Submission

Submit the following:

- Screenshots of the images before and after applying each filter.
- The code for each task.
- A brief report discussing how each filter affects image sharpness, blurring, and enhancement.